IST 387: Introduction to Applied Data Science
Lab 7: Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

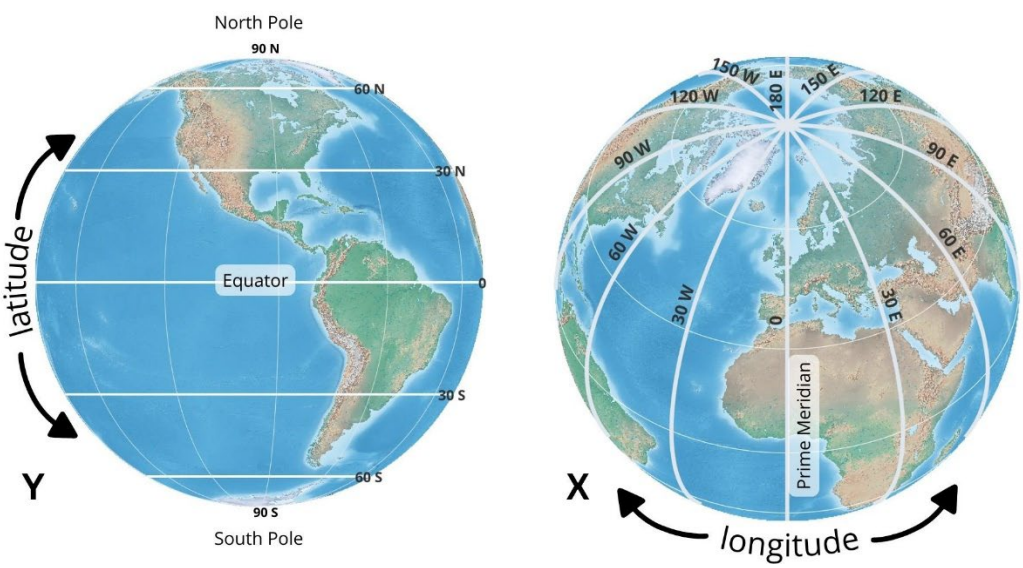
- Creating Map Visualizations with GGPLOT2
- Showing Points on a Map
- Important R Functions

Creating Map Visualizations with GGPLOT2

Map visualizations are similar in concept to the visualizations we previously discussed, but with the added geographical component of the displayed information. One important concept to be considered is color coding: i.e. how to use color and sill accommodate the many people who have one or another form of color blindness. One approach is to use a gray scale (a range from white to black). Another approach is to pick a color range that varies intensity from light to dark (such as ranging from white to light blue to dark blue).

Le's start with generating a simple map using the **ggplot package**. We do this in a similar fashion to other ggplot charts. Before we get started, we need to review a little geography - specifically the meaning of **longitude** and **latitude**.

As shown in the diagram below on the left, **latitude lines** are horizontal and parallel full circles, where 0 is the equator; +90 degrees is the North Pole, and -90 degrees is the South Pole (latitude lines are full circles).



As shown on the right side of the diagram, longitude lines are vertical arcs: they are not parallel and O represents the prime meridian in Greenwich, UK; +60 degrees is east and splits Europe from Asia; -60 degrees is west and passes through middle of south America. Longitude lines are half circles and that is why the overall range is ±180.

We can create the graphic above with the help of the **maps package** and the **mapproi package**. Although you can make simple maps with these packages, we really need the **ggplot2** package to make more sophisticated, data-based maps. Luckily, there is a companion package that helps **ggplot()** with mapping, known as **ggmap**. Once that is installed, we can get the map of the United States with the following line of code:

```
us <- map_data("state")
glimpse(us)
```

The output is as follows:

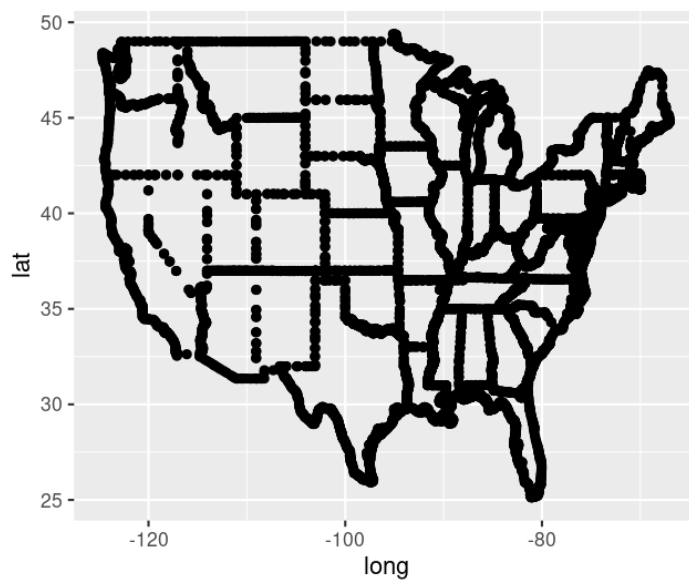
```
Rows: 15,537
Columns: 6
$ long      <dbl> -87.46201, -87.48493, -87.52503, -87.53076, -87.57087, -87.58806, -87.59379, -87.59379, …
$ lat       <dbl> 30.38968, 30.37249, 30.37249, 30.33239, 30.32665, 30.32665, 30.30947, 30.28655, 30.27509…
$ group     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1…
$ order     <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 2…
$ region    <chr> "alabama", "alabama", "alabama", "alabama", "alabama", "alabama", "alabama", "alabama", …
$ subregion <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, …
```

Here, the attribute **long** specifies the longitude (x location) and **lat** specifies latitude (y location).

We can see that **us** is a data frame of points, and we can visualize the points with the following **ggplot( )** code:

```
ggplot(us) + geom_point(aes(x=long, y=lat))
```

The output is as follows:



Yup, it is a map of the United States, but it is weird looking, for two reasons:

1. Those dots are obviously defining the borders of the states: we would much rather have thin, continuous lines instead of those big dots.

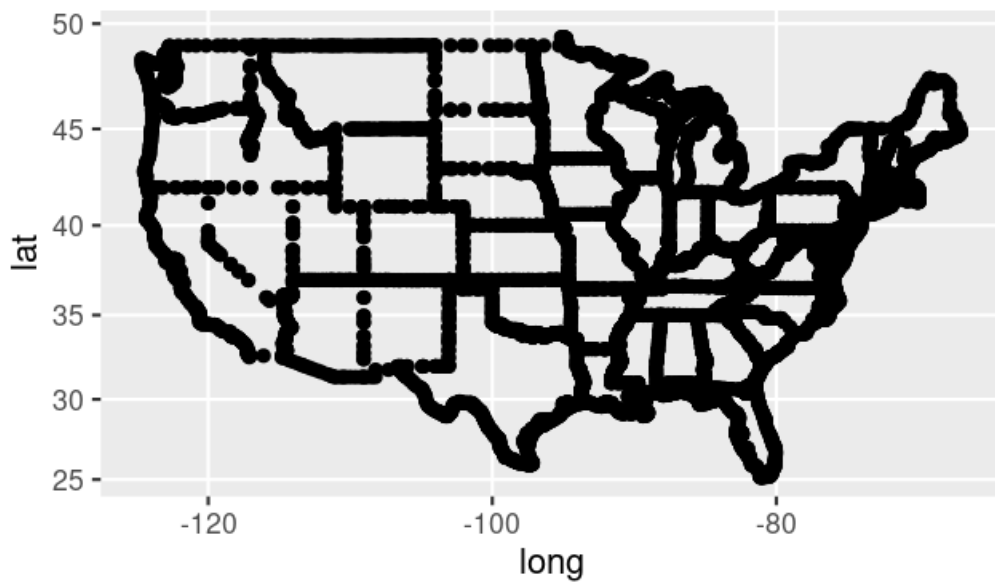
2. The map looks squashed because R has adjusted the calibration of the axes to make the result a square. The X-axis has two tick marks covering 20 degrees between each axis label, whereas the Y-axis has two tick marks covering just 5 degrees between labels.

In addition, the graphic above does not account for the curvature of the earth. Thus, we use a technique called a **map projection** to render a spherical area onto a flat surface. All map projections create some kind of visual distortion because a spherical object just does not want to

be flat! So, whenever we use **ggplot( )** to render a map where the curvature of the earth is large enough to matter, we need to choose a map projection based on the intended purpose of the map. We can choose the appropriate projection by using the **coord_map** function, which makes sure to project the map in the way we would like. For example, we can specify **coord_map (projection = "mercator")**, to use the Mercator projection. Note that **Mercator** is the default choice for the **coord_map** function, so we can simply use the following code to generate a map with a suitable projection:

```
ggplot(us) + geom_point(aes(x=long, y=lat)) + coord_map()
```

The output is as follows:

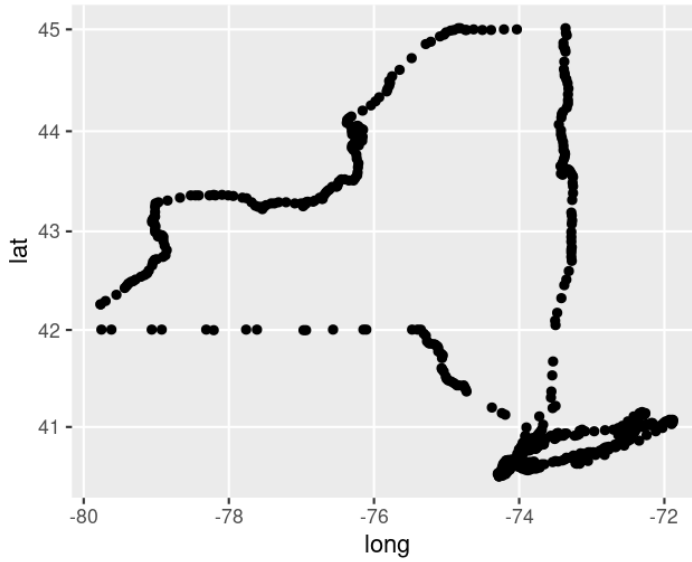


That U.S. map looks a lot better than before, although we still have thick dots instead of thin lines. We will fix that a little later, but for now, let's look at the other fields in the **us** datafame.

The **region variable** defines all the points that belong, with a given state. Let's break out one state to show how this works: in the code below, we filter the **us** data to create a subset of the data where the region value is equal to New York. We will store this in a new data frame called **NYData**. Then in the subsequent **ggplot**, we show of the points in the **NYData** data frame:

```
#zoom into NY
NYData <- us %>% filter(region=="new york")
ggplot(NYData) + geom_point(aes(x=long,y=lat)) + coord_map()
```

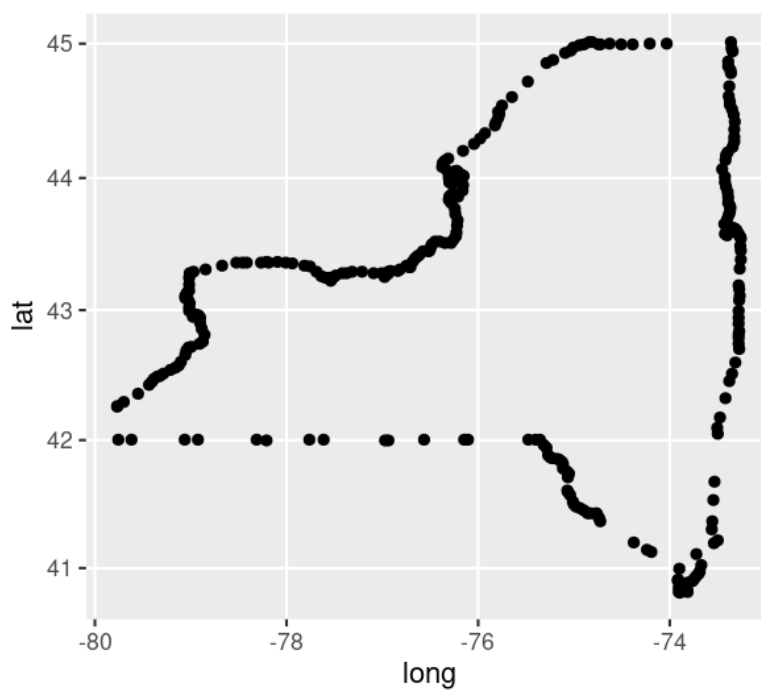
The output is as follows:



When drawing a map, there is often more than one "chunk of land" that is part of the state. For example, Manhattan is an island, so it is different landmass than upstate New York. The "group" attribute notes these different "chunks of land." With a bit of playing around, we can look at one of the groups in New York:

```
NYGroupLevelData <- us %>% filter(group==35)
ggplot(NYGroupLevelData) + geom_point(aes(x=long,y=lat)) + coord_map()
```

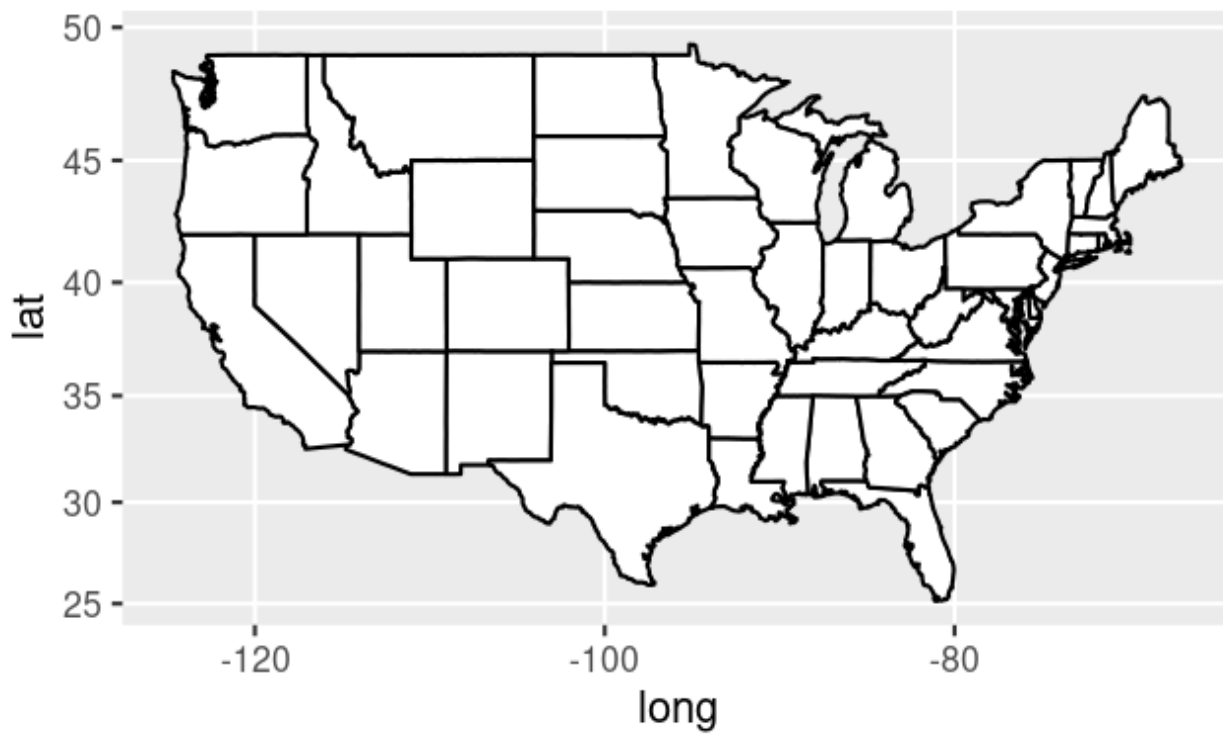
The output is as follows:



Now we are ready to improve the look of our map by plotting lines instead of dots. For this task, we will use the **geom_polygon( )** command. This command takes a list of dots and connects them together in the proper sequence to make a closed shape.

```
#generate the map using lines
ggplot(us) + geom_polygon(color="black",
    fill="white",aes(x=long,y=lat, group=group)) +
    coord_map()
```

The output is as follows:



Now we are getting somewhere! A simple figure like this is often called a **base map**. The map becomes much more interesting, however, when we add data to it. For example, we can shade each state, based on some attribute, such as the population of the state. To do this mashup, we can read in the state population dataset in the same way we have done in previous lab notes, and make sure the state names are all lowercase to be consistent with the region attribute in the **us** dataframe, which is lower case. After reading in the dataset, we then combine the population information with the us geometry information, using the merge command:

```r
library(tidyverse)
#read in the census data set (previous lab notes)
readCensus <- function(urlToRead) {

  #read the data from the web
  testFrame <- read_csv(url(urlToRead), col_names = FALSE)

  #use pipes to keep updating testFrame
  testFrame <- testFrame %>%

    #only keep the first 5 columns
    select(1:5) %>%

    #Delete unwanted rows
    slice(-1:-9) %>%
    slice(-54:-58)

    #Convert chr columns to numeric
    testFrame$X2 = as.numeric(gsub(",", "", testFrame$X2))
    testFrame$X3 = as.numeric(gsub(",", "", testFrame$X3))
    testFrame$X4 = as.numeric(gsub(",", "", testFrame$X4))

    #rename all columns of dataframe
    colnames(testFrame) <- c("stateName", "april10census", "april10base", "july10pop", "july11pop")

    #Remove Nas from dataframe
    testFrame <- na.omit(testFrame)

    #Remove dot from state name
    testFrame$stateName = gsub("\\.", "", testFrame$stateName)

    return(testFrame)
}
```

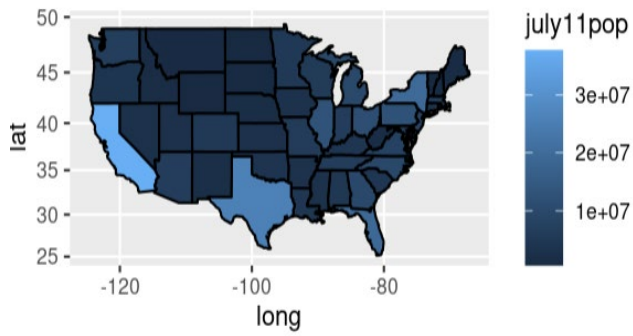
```r
us <- map_data("state")
# Use the readCensus function defined above
url = "http://www2.census.gov/programs-surveys/popest/tables/2010-2011/state/totals/nst-est2011-01.csv"

dfStates <- readCensus(url)

#make sure everything is lowercase
dfStates$state <- tolower(dfStates$stateName)

#combine dataframes, using state and region to merge
dfStatesWithGeom <- merge(dfStates, us, by.x="state", by.y="region")

#restore the order for the points in the polygon
dfStatesWithGeom <- dfStatesWithGeom %>% arrange(order)
```

The last line uses the **order** attribute, which is from the "**us**" dataframe, that has not yet been discussed. It defines the order of the points to draw a polygon. Sometimes, when doing a merge, the order of the rows gets lost. However, for our **geom _polygon** command, we need to make sure the points are in the order specified by the order attribute. Having done all this work, now we are finally able to create a map, with the fill color representing the population of each state. We do this by telling **ggplot** to fill each state based on the **july11pop** column of data. Everything else is the

same as the simple map we previously created. The technical name for this type of visualization is a **choropleth map**, but many people call it a "**heat map**," since this type of visualization can be used to show temperature forecasts on a weather map:

```
myMap <- ggplot(dfStatesWithGeom) +
  geom_polygon(color="black",
               aes(x=long, y=lat, group=group, fill=july11pop)) +
  coord_map()
myMap
```

The output is as follows:



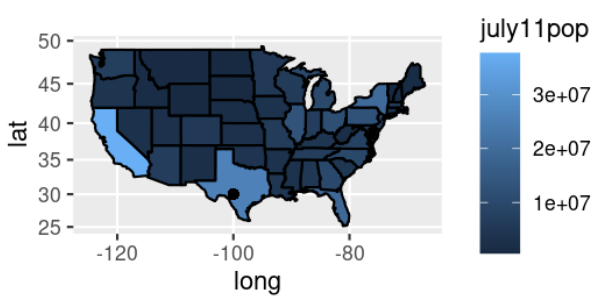
Note that we created a variable to store the map visualization ("myMap"). This R object is stored and ready to use for future visualizations.

Showing Points on a Map

Let's now add some points to the map. We can start by just hard coding the latitude and longitude of a specific location, for example, we can create a made-up point somewhere in Texas:

```
myMap + geom_point(aes(x = -100, y = 30))
```

The output is as follows:



Now southern Texas contains a little black dot. We used **geom_point** to create that single point, specified by x (longitude) and y (latitude) within the **geom_point( )** function.

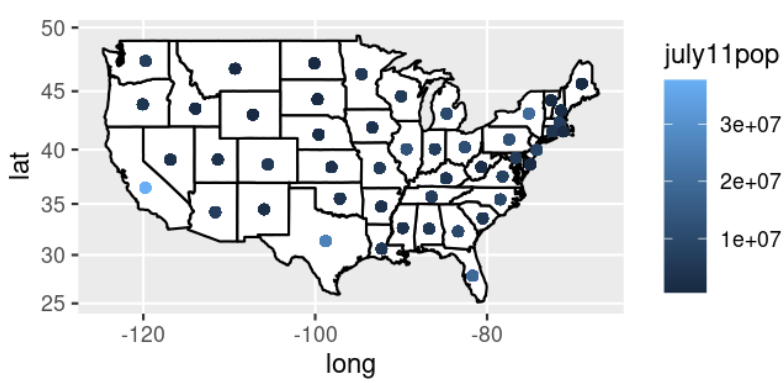
Next, let's show data as points on the map! For example, we can put a point at the center of each state and have the color (or size) represent the population. To do this, we note that **gmap** provides the list of state names available as a variable **(state.names),** as well as the centers of each state (**state.center**). So, we create a simple little **stateCenter** dataframe, and then merge that with our existing larger dataframe. Note that, as before, the states should be in lower case, just to be consistent across dataframes (when we try to merge), which is why we used the **tolower** command:

```
# create a dataframe  with state centers and population
stateCenter <- data.frame(state=tolower(state.name),
                          x=state.center$x,
                          y=state.center$y)

#merge with the dataframe we have been using
dfStatesWithCenter <- merge(dfStatesWithGeom,
                            stateCenter, by="state")

#don't forget to sort on the order of attribute
dfStatesWithCenter <- dfStatesWithCenter %>% arrange(order)
```

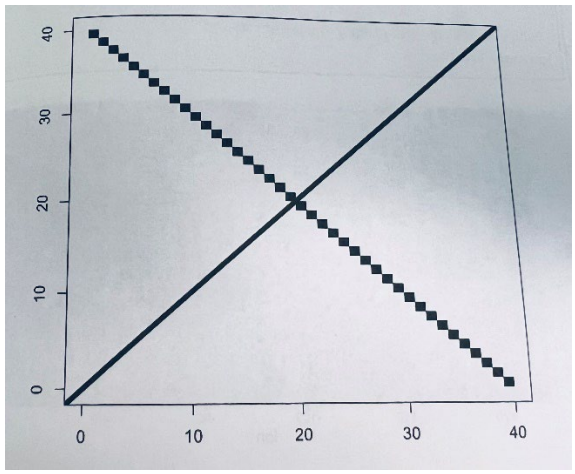
As you can see, the merge function creates a new dataframe by identifying common columns (via the named column "state"), and then combining the different attributes (columns). Now that we have our desired dataframe, creating the visualization is like what we have previously done but this time showing the center of each state:

```
#add a points layer with color=population
ggplot(dfStatesWithCenter) +
  geom_polygon(color="black", fill="White",
               aes(x=long, y=lat, group=group)) +
  geom_point(aes(x=x,y=y,color=july11pop)) +
  coord_map()
```

The output is as follows:



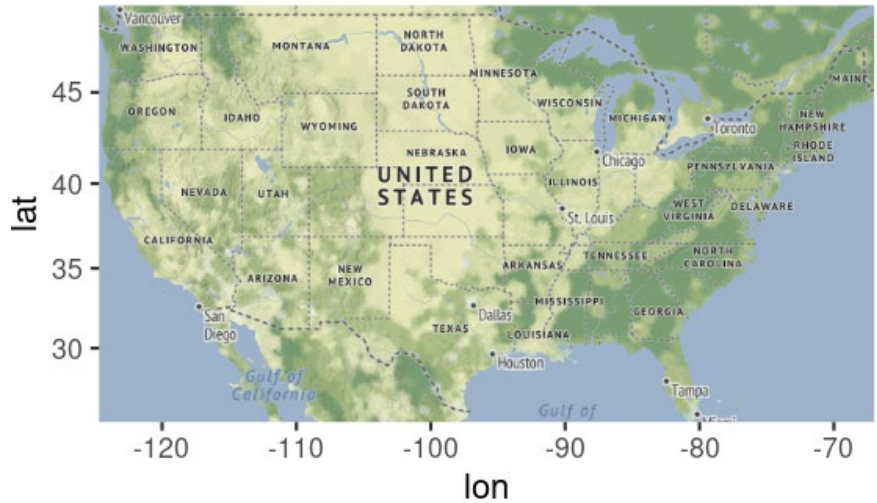
We can make the map more interesting by putting an image of a map "behind" our points and outline of states. Up until this point, all our visualizations have been with vector graphics, which works by drawing lines between points (or drawing markers at a point. However, as shown in the figure below, **raster graphic** treats every image as a grid of pixels rather than a set of smooth lines. In the figure below, the diagonal line that goes from the top left to the bottom right is a **raster drawing,** whereas the one that goes from lower left to upper right is a **vector drawing**. Notice that the raster line has "**jaggies**" because each individual dot is visible.



Our new map, behind the vector graphics, will be a **raster image**, which can include terrain information. To create a background **raster image** map for our visualization, we can use the **get_stamenmap(),** which is another function in the **ggmap** package. **Stamenmaps** are "**tiled**" images of varying levels of detail (the higher the zoom number, the more detail). In addition to the level of zoom (detail), we also need to provide the range of longitude and latitude (i.e. the bounding box of the map image, in longitude and latitude). Below we use all the longitude and latitude points in our dataframe to determine the bounding box for our image. For more information on stamen maps, you can check out the website of the company that created them (http://maps.stamen.com).

```
#put a map image behind the visualization
bb <- c(left = min(dfStatesWithCenter$long),
        bottom = min(dfStatesWithCenter$lat),
        right = max(dfStatesWithCenter$long),
        top = max(dfStatesWithCenter$lat))

#actually get the map
# note if the zoom is too large, it will take a long
# time to load the maps across the internet
install.packages("ggmap")
library("ggmap")
register_stadiamaps(key = "f0c40ccd-cd2e-4571-87ed-2700a0e109ad", write = TRUE)
map <- get_stadiamap(bbox = bb, zoom=4)

ggmap(map)
```

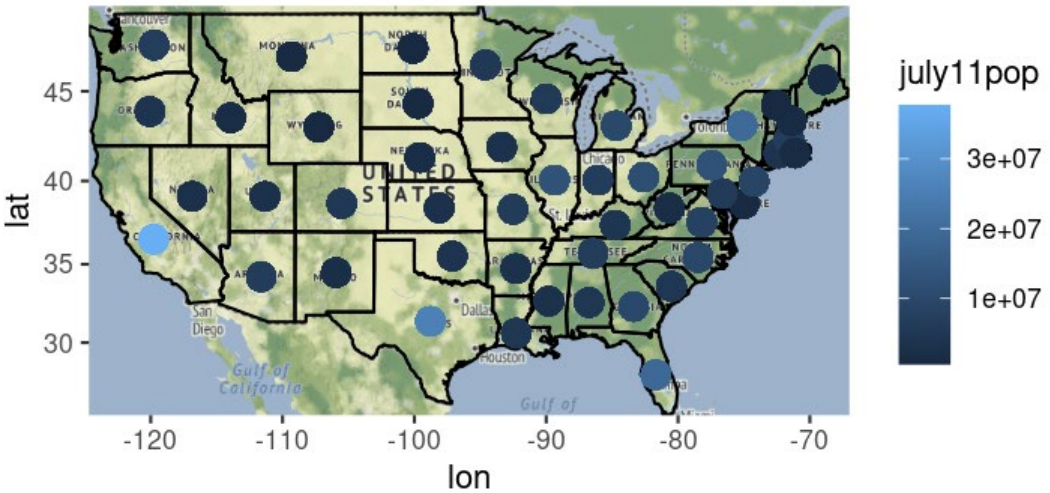
The output is as follows:



Cool! This shows a map as an image-and now we need to put data (via points) onto the map, which we can do via the **geom_point** command, which can be a layer within the **ggmap**, just like **geom_point** can be a layer when using the **ggplot( )** command.

```
# add a points layer with color=population
ggmap(map) +
  geom_polygon(data = dfStatesWithCenter,
               color="black", fill="NA",
               aes(x=long,y=lat,group=group)) +
  geom_point(data = dfStatesWithCenter, size=4,
             aes(x=x,y=y, color=july11pop))
```

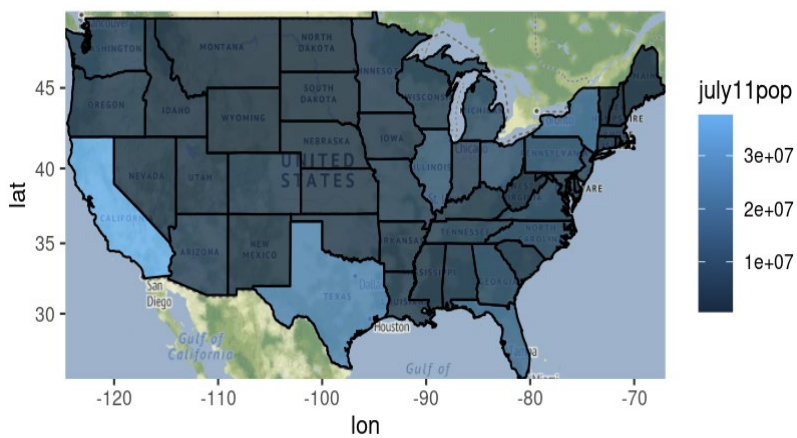
The output is as follows:



Rather than showing the population as points, we can go back to color coding the states to be based on population, but make each state somewhat transparent, so we can see the map background as well as the color. Within the **geom_polygon** command, we can specify alpha, which ranges from 0 (fully transparent) to 1 (fully opaque). The code for generating this visualization is shown below:

```
ggmap(map) +
  geom_polygon(data=dfStatesWithGeom,
               color="black", alpha=0.8,
               aes(x=long, y=lat, group=group, fill=july11pop))
```

The output is as follows:



Important R Functions

- **c( ):** Combine elements to make a vector.
- **coord_map( ):** Makes sure the map does not show up stretched or distorted.
- **data.frame( ):** Creates a dataframe.
- **filter( ) :** Tidyverse function to select rows.
- **fromJSON( ):** Convert from JSON format into a data structure.
- **geom_point( ):** Adds points to a visualization.
- **geom_polygon( ):** Adds filled polygons to a visualization.
- **get_stamenmap( ):** Gets a raster image map.
- **ggmap( ):** Creates a map visualization.

- **ggplot( ):** Creates a ggplot.
- **ggtitle( ):** Adds a title to a visualization.
- **group_by( ):** Tidyverse function to organize data into groups.
- **map_data( ):** Finds a map to be used by ggplot.
- **max( ):** Returns the maximum value from a vector.
- **merge( ):** Joins two data sets according to a matching key.
- **mutate( ):** Tidyverse function to modify columns.
- **str_replace_all( ):** Multiple string substitution.
- **summarize( ):** Tidyverse function to create column summaries.
- **tolower( ):** Changes all uppercase letters to lowercase,